

SQLAlchemy Constraints

 <https://github.com/learn-co-curriculum/python-p4-validation-constraints>  <https://github.com/learn-co-curriculum/python-p4-validation-constraints/issues/new>

Learning Goals

- Define constraints and validations in data processing.
- Restrict database input to specific formats using constraints.

Key Vocab

- **Constraint:** a rule enforced on the data columns of a table. Ensures that only appropriate data is saved to the database.
- **Validation:** an automatic check to ensure that data entered is sensible and feasible.
- **Forms:** A web form (or HTML form) is a place where users enter data or personal information that's then sent to a server for processing.

Introduction

Invalid values in our database can cause all kinds of problems when we want to use the data. To keep invalid values out of our database we can use SQLAlchemy constraints. These constraints work at a database level to control the data we add to the database.

unique and nullable Constraints

Certain constraints in SQLAlchemy can be added directly to the column as we define it. The `nullable` constraint allows us to make sure the values added to the columns are not null. We can define this by adding the argument `nullable=False` to the `Column` constructor.

If it's required that the values in your columns must be unique then the `unique` constraint can be specified by adding the argument `unique=True` to the `Column` constructor.

CheckConstraint

CheckConstraints can be created for columns and tables. The text of the CheckConstraint is passed directly through to the database. Some databases like MySQL do not support CheckConstraints.

Here is an example of a Patient table that uses constraints to control input of `birth_year` and `death_year`.

```
class Patient(db.Model):
    __tablename__ = 'patient'
    name = db.Column(db.String, primary_key=True)
    # Constraint defined at the Column level
    birth_year = db.Column(db.Integer,
                           db.CheckConstraint('birth_year < 2023'),
                           nullable=False)
    death_year = db.Column(db.Integer)
    # Constraint defined at the Table level
    __table_args__ = (
        db.CheckConstraint('(death_year is NULL) or (death_year >= birth_year)'),
    )
```

Here we have a column CheckConstraint which makes sure we cannot add a `birth_year` that is greater than 2023. We also have a table CheckConstraint that makes sure the death year is after the birth year or the death year is null (meaning that the patient is still alive).

Conclusion

SQLAlchemy constraints allow us to control input at a database level. They allow us to use any valid SQL and numeric comparisons to validate input. In the next lesson we will learn how to control input at a Python ORM level.

Resources

- [Defining Constraints and Indexes - SQLAlchemy](https://docs.sqlalchemy.org/en/14/core/constraints.html)  (<https://docs.sqlalchemy.org/en/14/core/constraints.html>)